Shelf Voice — 04 Testing and Verification

Oct 27, 2026 · @Aradhya Chaudhary

Status and open decisions

Version 1.0, approved by the owner on Oct 27. All six testing decisions are final; changes from here go through this table.

#	Decision	Options (recommendation %)	Why the top option	Owner's choice
V1	End-to-end test framework	Playwright (85%); Cypress (15%)	Fakes the microphone with a WAV file in Chromium, runs mobile viewports, free parallel runs in CI	Playwright
V2	AI evaluation tool	promptfoo (65%); custom Vitest eval scripts (35%)	Ships a Meta Model API provider, JSON-schema and tool-call assertions, and a prompt-injection red-team module	promptfoo
V3	Meta API calls in CI	Recorded responses on every PR, live eval suite nightly and on demand (70%); live calls on every PR (20%); mocks only (10%)	Fast, free, deterministic PR checks; real-model drift still caught daily	Recorded on PRs, live nightly
V4	Coverage policy	80% lines on server services and forecast code, no global target (65%); 70% global (35%)	Coverage where bugs cost most; no time wasted testing UI markup for a number	80% on server + forecast code
V5	Voice test set source	Team records 30+ clips plus a few from real owners with consent (60%); team only (40%)	Real counter speech and noise are what the demo will face	Team + real owners with consent

#	Decision	Options (recommendation %)	Why the top option	Owner's choice
V6	Performance testing	Lighthouse CI on storefront + small k6 load test on review endpoint (60%); Lighthouse only (40%)	Proves both the 3-second storefront target and that rate limits hold under load	Lighthouse CI + k6

Testing strategy and test levels

Most tests are fast unit and database tests that run on every pull request; AI quality is measured separately against fixed test sets, because model output varies and needs scores, not pass/fail on exact text.

A feature counts as done only when its tests pass and the code has been read against the spec ("reading": source, API contract, schema and test results inspected before calling it correct).

Level	What it proves	Tool	Location	Runs
Unit	Forecast maths, Zod schemas, parsers, tool-argument validation, WAV encoder	Vitest	`src/**/*.test.ts`	Every PR
Integration	Service functions against a real local Postgres; AI module against recorded Meta responses	Vitest + Supabase local stack	`tests/integration`	Every PR
API contract	Each route's status codes, error shape and response schema	Vitest calling route handlers	`tests/api`	Every PR
Database and RLS	Every policy isolates shops; RLS enabled everywhere	pgTAP via `supabase test db`	`supabase/tests`	Every PR

Level	What it proves	Tool	Location	Runs
AI evaluation	Extraction accuracy, command correctness, review analysis, injection resistance	promptfoo (V2)	`evals/`	Nightly and on demand, live Meta API
End-to-end	The four PRD journeys in a real browser with a fake microphone and camera	Playwright (V1)	`e2e/`	Every PR (recorded AI) and before release (live AI)
Accessibility	WCAG 2.1 AA rules on key pages	axe-core in Playwright	`e2e/a11y`	Every PR
Performance	Storefront load, review endpoint under load	Lighthouse CI, k6 (V6)	`perf/`	Before release
Security	Secrets, dependencies, code patterns, injection set	GitHub secret scanning, npm audit, Semgrep, promptfoo	CI	Every PR

Test-driven rule from the TDD skill: for every bug, first write a failing test that reproduces it, then fix it.

Database and authorization tests

Every RLS policy is tested by signing in as one owner and failing to reach another owner's rows; these tests run on every pull request and block merges.

Setup: pgTAP with the Basejump Supabase test helpers, which create test users and switch the session to each user inside a rolled-back transaction. Tests run with `supabase test db` against the local stack.

Test	Expectation
RLS enabled on every table in `public`	One helper call fails if any table lacks RLS

Test	Expectation
Owner A selects shop B's products, movements, proposals, reviews	0 rows returned
Owner A inserts a product with shop B's ID	Rejected
Owner A updates or deletes shop B's rows	0 rows affected
Anonymous visitor reads the public storefront view	Only published shops, only safe columns
Anonymous visitor reads base tables	0 rows
Demo user reaches the original demo shop	0 rows; only their own copy
Storage: owner A reads or writes under shop B's folder	Denied
Storage: anyone reads catalog-images	Allowed; voice and photos buckets denied
Owner hides a review	Allowed; owner edits review text: denied
Delete shop (FR-16)	All rows for that shop gone; other shops untouched
Stock never negative	Check constraint rejects it

API-level authorization (Vitest against route handlers): no session gives 401, a foreign shop ID gives 404 (not 403, so IDs cannot be probed), a body for another shop's product gives 404, and proposal confirm is idempotent (a second confirm changes nothing).

AI evaluation

Each AI job has a labelled test set and a pass threshold; speech and understanding are scored separately so a failure points to the right model.

Two-stage scoring: (1) transcription quality on audio, then (2) extraction or command quality on hand-corrected gold transcripts. The end-to-end score on audio is reported too, but stage 2 is what prompt changes are judged on.

Test set	Size	Metric	Pass threshold
Transcription: owner and customer clips (Hindi, English, Hinglish, with shop noise)	30+ clips	Product names transcribed correctly, with vs without keyword hints	At least 90% with hints; report the improvement over no hints
Shop setup extraction	30 transcripts	Product recall and precision on name + price + unit	Recall at least 90%; precision at least 95%
Invented values	Same 30	Prices or hours filled in that were never spoken	0
Catalog and sales commands	40 transcripts with expected tool calls	Correct tool and arguments	At least 90%
Ambiguous commands ("Maggi" when there are two Maggi items)	10 transcripts	Model asks for clarification instead of guessing	100%
Bill reading	15 photos (10 printed, 5 handwritten)	Line items with correct quantity	Printed at least 85%; handwritten reported, no gate
Review analysis	30 recorded reviews, labelled by 2 teammates	Sentiment, items mentioned, stock-out flag agree with labels	At least 85% overall; stock-out recall at least 90%
Prompt injection	20 adversarial review and command transcripts	Unexpected tool calls or schema breaks	0
Restock explanations	20 forecast results	Sentence quotes the computed numbers exactly, in the requested language	100% numbers match (checked by regex)

How it runs (V2, V3): promptfoo configs in `evals/` use Meta's provider against muse-spark-1.3, with JSON-schema and tool-call assertions plus custom JavaScript scorers for recall and precision. Results are saved per run, so a prompt change shows its score

difference before merge. Latency and cost are asserted too: extraction under 20 s at p95, average cost per call under $0.02.

Forecast backtest

The forecast is judged by replaying 90 days of sales day by day and counting how many stock-outs it warned about at least 2 days ahead; the PRD target is 80%.

Data:

- A seeded generator (fixed random seed, so results repeat) creates 90 days of sales for 40 typical kirana items: a base daily rate per item, a weekend lift, a festival spike, random noise, and a few slow sellers with many zero days.
- If any of the 3 real shops share sales logs with consent, their data is replayed too and reported separately.

Method (rolling origin):

1. Start on day 30 with stock set to two weeks of demand.
2. Each day, compute the forecast using only data up to that day, apply the reorder rule from document 02, and deliver orders after the lead time.
3. Record each day: warnings raised, actual stock-outs, units on hand.

Metrics and gates:

Metric	Gate
Stock-outs warned at least 2 days ahead	At least 80%
Stock-out days vs "reorder when empty" baseline	At least 50% fewer
Average units on hand vs a 4-week moving-average policy	No more than 10% higher
Daily demand error (mean absolute error)	Reported per item type

The backtest is a Vitest suite (`tests/forecast/backtest.test.ts`), runs in under 10 seconds, and is how the smoothing factor and multipliers in document 02 are tuned.

Voice and camera testing

Voice flows are tested end to end by feeding a WAV file to the browser as if it were the microphone, so the real recorder, encoder, upload and transcription all run.

Fake microphone (Chromium in Playwright): launch with `--use-fake-device-for-media-stream`, `--use-fake-ui-for-media-stream` and `--use-file-for-fake-audio-capture=<clip>.wav`. The file is fixed per browser launch, so each voice scenario runs in its own browser context. Firefox and WebKit cannot inject a file, so voice journeys run on Chromium only; other browsers get the non-voice journeys.

Audio fixtures (`tests/fixtures/audio/`):

Fixture group	Count	Content
Shop setup	6	30-90 s descriptions with 10-20 products, Hindi, English, Hinglish
Commands	12	Price change, out of stock, sale, delivery, ambiguous item
Reviews	10	Positive, negative, stock-out complaint, silence, background noise only
Edge cases	6	1 s clip, 16 s clip (over limit), clipped loud audio, 44.1 kHz file, silence, a spoken injection attempt

All clips are recorded by the team, or by real owners who agree in writing (V5); no clip contains names or phone numbers.

Unit tests for the in-browser WAV encoder check the 44-byte header, mono 16-bit PCM, 16 kHz rate and correct length for a known sine-wave input.

Camera: Playwright uploads fixture photos through the file input (`setInputFiles`), since camera capture uses the same input. Fixtures: 5 shelf photos, 10 printed and 5 handwritten bills, 1 photo with GPS EXIF data to prove location is stripped.

Manual device check before release: 2 real Android phones (one budget model) record setup, command and review clips in a real shop setting.

End-to-end, accessibility and performance tests

Five end-to-end scenarios cover the PRD journeys; they run at a 360 px phone viewport and a 1280 px desktop viewport.

Scenario	Steps	Must see
E2E-1 Shop setup	Demo sign-in, record setup clip, review proposals, fix one price, publish	Storefront shows every confirmed product; QR downloads

Scenario	Steps	Must see
E2E-2 Customer browse and order	Open storefront link on phone viewport, filter a category, tap order	WhatsApp link with prefilled item list and shop number
E2E-3 Voice review	Open review page from QR URL, accept consent, record stock-out clip	Thank-you within 5 s; owner feed shows the review flagged; item rises in restock list
E2E-4 Log and restock	Record a sale clip and a bill photo, confirm both	Stock changes match; restock list shows quantity and reason
E2E-5 Delete shop	Settings, delete, confirm	Signed out; storefront returns 404

Accessibility: axe-core runs on the storefront, review page, owner dashboard and proposal review screen, with zero serious or critical violations allowed. Manual checks: keyboard-only pass through the proposal screen, screen-reader labels on the record button, and colour contrast of status badges.

Performance (V6):

Test	Tool	Target
Storefront, mobile profile, throttled 4G	Lighthouse CI	Performance score at least 90; LCP under 2.5 s; CLS under 0.1; JavaScript under 100 KB
Owner dashboard, mobile profile	Lighthouse CI	LCP under 3 s
Review submit, 20 virtual users for 2 minutes	k6 against staging with AI stubbed	p95 under 1 s; rate limit returns 429 at the set thresholds; no 5xx
Voice note to proposal (live AI)	Timed in the eval run	p95 under 25 s

Failure scenarios and resilience

Every external dependency gets a test that makes it fail and checks the owner sees a clear message and loses no data.

Failure injected	How	Expected behaviour
Meta API timeout	Stubbed client delays past 30 s	One retry, then "Couldn't understand that right now, your recording is saved"; Retry button reuses the stored clip
Meta API 429 or 5xx	Stub returns error	Backoff and one retry; then same message; ai_calls row marked FAILED
Meta returns invalid JSON or out-of-schema output	Recorded bad response	Zod rejects; no proposal saved; error logged with call ID
Model proposes a product ID from another shop	Crafted response	Server drops it; owner sees "couldn't match"; test asserts no write
Transcription returns empty text	Silent clip	"We didn't hear anything, try again"
Upload interrupted	Playwright goes offline mid-upload	Clip kept in IndexedDB; uploads automatically when back online
Supabase unreachable	Wrong URL in test env	Storefront serves cached page; owner app shows a connection error, not a blank screen
Turnstile fails or is missing	No token sent	Review rejected with 403 and a friendly message
Daily AI budget reached	Seed the counter at the cap	AI features paused with a message; non-AI features keep working
Duplicate confirm (double tap)	Two confirm requests	Second is a no-op; stock changes once

Test data, fixtures and mocking

Tests never touch production data, and the Meta API is mocked at one seam: the AI client module.

Asset	Where	Used by
SQL seed: 2 owners, 2 shops, 40 products, 90 days of generated sales, 10 reviews	`supabase/seed.sql` (generated by `scripts/seed.ts` with a fixed seed)	Local dev, integration, E2E, demo shop
Demo shop seed	`scripts/demo-shop.ts`	Production demo sign-in and nightly reset
Audio and photo fixtures	`tests/fixtures/`	E2E, evals
Gold transcripts and labels	`evals/data/*.jsonl`	promptfoo evals
Recorded Meta responses	`tests/fixtures/meta/*.json`	Unit, integration, PR E2E

Mocking rules:

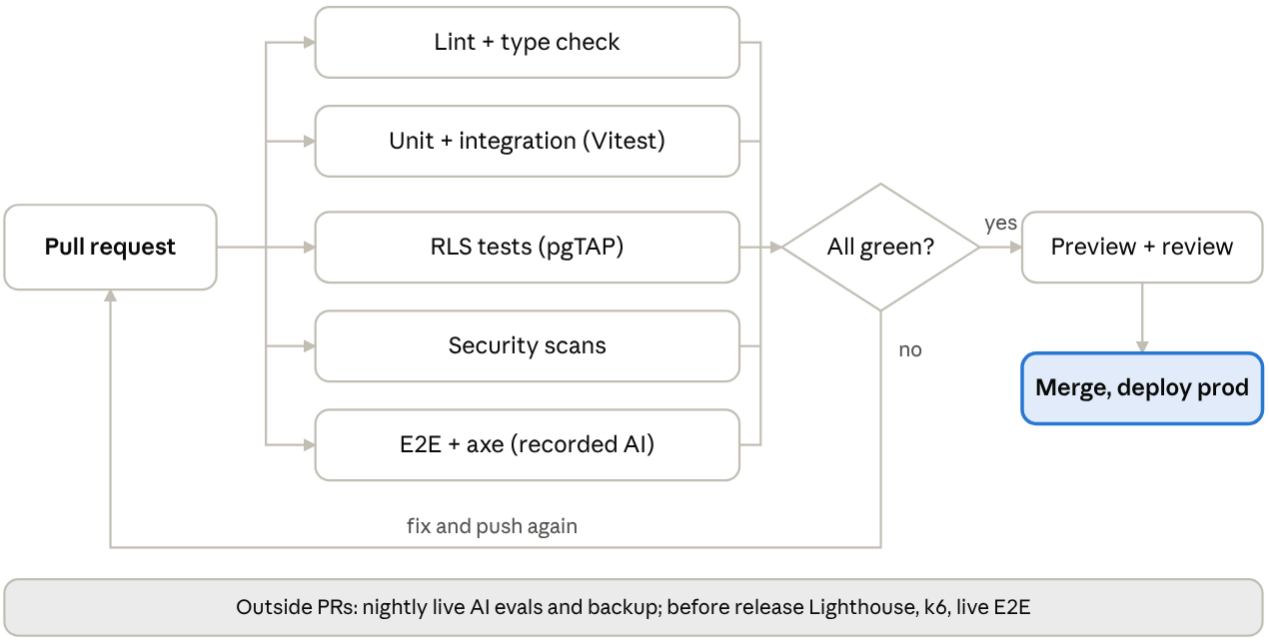
- Only `src/server/ai/meta.ts` is mocked. A test-mode switch makes it return recorded responses keyed by a hash of the request, so tests are deterministic and free.
- A script (`pnpm record:meta`) refreshes the recordings from the live API when prompts change; recordings are reviewed in the pull request like code.
- Supabase is never mocked: integration and RLS tests use the real local stack, because mocked databases hide policy bugs.
- Time is frozen in forecast and retention tests.

Coverage (V4): reported on every PR; the merge is blocked if line coverage of `src/server/**` and the forecast module falls below 80%. UI components are covered by E2E, not by a coverage number.

CI quality gates

Five checks run in parallel on every pull request in GitHub Actions; any failure blocks the merge, and the live-AI and performance checks run outside pull requests so PRs stay fast and free.

A pull request merges only when all five checks pass



CI pipeline · 5 parallel checks, 1 gate

Blocking on every PR: lint and TypeScript strict type check, Vitest unit and integration tests, pgTAP RLS tests, security scans (npm audit with no high or critical in production deps, Semgrep, secret scan, bundle key check), and Playwright E2E with axe using recorded AI responses. Coverage is reported per V4. Nightly: live promptfoo evals and the production backup. Before release: Lighthouse CI, k6 and E2E with live AI.

Acceptance traceability and release verification

Every PRD story and success metric maps to at least one test; a story is accepted only when all its tests pass.

PRD item	Proven by
Story 1: shop setup by voice (FR-02 to FR-04)	E2E-1; setup extraction eval; Lighthouse storefront test
Story 2: voice catalog edits (FR-05)	Command eval; ambiguous-command eval; E2E-4
Story 3: voice sales and deliveries (FR-06, FR-11)	Command eval; bill eval; stock constraint test; E2E-4
Story 4: restock list and reasons (FR-07, FR-09)	Forecast backtest; explanation eval; E2E-3 rise check

PRD item	Proven by
Story 5: customer voice review (FR-08)	E2E-3; Turnstile and rate-limit tests; k6 load test
Story 6: review insight (FR-09)	Review analysis eval; injection eval
FR-01 sign-in and demo sign-in	E2E-1; RLS demo-copy test
FR-16 delete shop	E2E-5; delete-shop RLS test
Metric: under 3 minutes to storefront	Timed manual run with a real owner, recorded
Metric: 3 real owners tried it	Signed consent + recording per owner

Release verification (before the submission build and the demo video):

1. All CI gates green on the release commit.
2. Live eval suite run against production model settings; every threshold met or a signed-off exception noted.
3. Security pre-production checklist in document 03 complete.
4. E2E suite run against the production URL with live AI, including one run on a real Android phone.
5. Demo shop reset and checked.
6. Results recorded in document 09 with one status per check: PASS, FAIL, BLOCKED, NOT RUN or NOT APPLICABLE. Nothing unexecuted is marked PASS.

Sources

Checked Oct 27, 2026.

- Playwright fake audio input: Chromium flags to feed a WAV file as the microphone; one file per browser launch
- Playwright fake media for Firefox and Chromium: no file injection on WebKit
- Supabase test helpers (Basejump): pgTAP helpers for test users and RLS checks, run with `supabase test db`
- Database testing with pgTAP: RLS isolation test patterns
- promptfoo guide 2026: JSON-schema, tool-call and custom assertions; red-team module
- promptfoo Meta provider: Meta Model API support